A Objective

This lab exercise will provide additional insights into security of Linux file systems, in particular with regards to file access control features. We will build upon the security setup of our little example organization (*Gotham City*) from the previous lab exercise.

B Refresh: Platform and process

- You will need a Linux-system where you have superuser (root) privileges (with `sudo` oder `su`) – please use the prepared Kali VM running (the privileged `sudo`-user is called `junioradmin`)!
- Execute all commands in a terminal window using the standard shell and don't forget to elevate your privileges first using e.g. `sudo su`!

→ Be sure to provide answers to all questions and to document **each and every** shell command line in your lab notes (lab report)! In particular:

- Provide all important command options you are using and always list the resulting permissions (for permissions on directories use e.g. `ls -ld dirname`)
- Don't forget that in order to use a directory "normally", you need to grant `r-x` permissions at minimum (to list contents, to `cd` to the directory, to access files in the directory)!

C Refresh: Creating users and groups

(1) If not already done, create the following users, groups and directories on the system (use the script `addusers.sh` you developed in lab "Linux User Management" or "Linux Benutzerverwaltung"!):

Note: If you already did this successfully in a previous lab, you can **skip** this step! Otherwise, use a simple shell script with one command per line!

Hint: See lab "Linux User Management" or "Linux Benutzerverwaltung"; *xx* are the last two digits of your login-number!

Username	Full Name	Login Group	Home Directory	Password
admin*NN*	*YourName*	heroes*NN*	`/home/heroes`*NN*`/admin`*NN*	`linux!is!c00l!`
batman	Bruce Wayne	heroes*NN*	`/home/heroes`*NN*`/batman`	`darkknight`
robin	Dick Grayson	friends	`/home/friends/robin`	`nightwing`
catwoman	Selina Kyle	friends	`/home/friends/catwoman`	`meowmeow`
gordon	James Gordon	police	`/home/police/gordon`	`law-n-order`
joker	The Joker	villains	`/home/villains/joker`	`whysoserious`
penguin	Oswald Cobblepot	villains	`/home/villains/penguin`	`fish!`

D Refresh: Creating Data Directories

You should have already been done the following three tasks during your previous lab-session:

(2) Create `/data26/*` directories for the groups `friends`, `police` and `villains` to use with corresponding names `/data26/friends`, `/data26/police` and `/data26/villains`. Owner in all cases is `root`.

(3) Members of the respective group shall have full access to their directory (they are also permitted to write), all others have *no access rights*.

(4) `batman` should nevertheless be permitted full control (including write permission) over these directories.

(5) Create a special directory `/data26/gotham` with `batman` having full access (including the permission to create, delete and rename files), but all other users having only read access to the directory (they may "use" the directory, but not write to it, i.e. they may *not* create new files, rename files or delete files).

E File Permissions – Fine Tuning

Solve all tasks by using the standard Unix/Linux access control system (you will not need ACLs e.g. the `setacl` command)!

(6) Check and, if required, correct the access permissions for the parent directory `/data26`: Owner should be `root` (full control), all others should only be permitted to read the directory contents and navigate into this folder.

(7) Change the permissions on the three group directories `/data26/*` (those private to each group) in such a manner that any new file created automatically receives the group of the directory itself (and not the group of the file creator)!

(8) Create a directory `/data26/transfer`. It will serve as a data exchange folder to easily share files between different groups of users and should permit the following access patterns:

i. Administrator of `/data26/transfer` shall be `gordon`: This user shall have full control of this folder and also be permitted to clean it up, i.e. delete files.
ii. All users should be able to create files in this folder, but they should not be able to delete any files they do not own.
iii. Any file that is created in this directory should automatically receive the (new) group `transfer`.

(9) Test this by creating the following file in each of the directories `/data26/*` when working as user `root`:

- filename is *xx*`.txt` (*xx* are the last to digits of your login-number again)
- *xx*`.txt` consists of the text "`This file has been created by` *Your name*`!`"
  Maybe you need to "mask" the "`!`"?

(10) How does `ls -l` display these refined permission settings *(Hint: STICKY BIT, SETGID BIT)*?

(11) Issue the command `sudo find /data26 -ls` and copy the output into your lab report.

F Testing our Security Setup

Verify the access control settings as described below. Check the following items for each:

- Note down precisely which commands you used!
- Take note in your lab report whether the prescribed action was successful (or not!) and *why* exactly!

  Is this the expected outcome?
- How did you change your user-id?

(12) As user `catwoman`: Create three files in `/data26/transfer`: `file1.txt` to `file3.txt`, each one with the corresponding content "`file`*x*`.txt by catwoman`". Create the text content of the files using the command line (**not** by using a text editor)! *What group are these files associated with?*
(13) As user `gordon`: Create two additional files `file4.txt` and `file5.txt` in `/data26/transfer`, each one with the content "`file`*x*`.txt by gordon`".
(14) As user `robin`: Copy `file1.txt` to `/data26/friends` and *then* delete the source file.
(15) As user `joker`: Create the file `playingcard.txt` in `/data26/villains`, then copy it to `/data26/transfer`.
(16) As `batman`: Copy `file2.txt` to `/data26/friends`, `/data26/police` and `/data26/villains`. Further, delete the file `playingcard.txt` in `/data26/transfer`.

(17) As `catwoman`: Delete the files `playingcard.txt` and `file1.txt` in `/data26/transfer`

(18) As `gordon`: Delete the files `file2.txt` and `playingcard.txt` in `/data26/transfer`, then move `file3.txt` to `/data26/police`.

(19) As `gordon`:

a) Create a **hard** link `just4me.txt` in `/data26/police` that points to the same file as `/data26/transfer/file4.txt`.

b) Change the permissions of `just4me.txt` in such a manner that only `gordon` can read and write this file (nobody else has permissions for accessing this file).

c) Test if `gordon` can indeed read the file using `cat`.

d) How did the access permissions of `/data26/transfer/file4.txt` change in the course of this operation?

e) Use the `stat` command to check the inode-information of these files – What is their inode-number? What value does their *link counter* have?

(20) As `gordon`:

a) Create a **soft** link `soft5.txt` in `/data26/police` that points to the same file as `/data26/transfer/file5.txt`.

b) Change the permissions of `soft5.txt` in such a manner that only `gordon` can read and write this file (nobody else has permissions for accessing this file).

c) Compare the file permissions of `soft5.txt` and `file5.txt` and explain!

d) Is the inode-number of the two files the same again? How can you determine the inode-number using just the `ls` command *(what option?)*?

e) What value does their *link counter* have now?

(21) As `gordon`:

a) Delete the files `file4.txt` and `file5.txt` in `/data26/transfer`.

b) What happened to the two links you created previously in `/data26/police` (`just4me.txt` and `soft5.txt`)? Can they still be used? Try to read each file using `cat` and explain!

(22) Issue the command `sudo find /data26 -ls` as user `root` and copy the output into your lab report.

(23) As `gordon`: Delete the directory `/data26/transfer` with all its contents!

We wish you lots of success and fun with the exercise!